

OBSERVACIONES DE LA PRÁCTICA

Sarah González Almario Cod 202521379
Andres Jose Velasco Mejia Cod 202610866
Juan José Peñaloza Ramírez Cod 202421523

Máquina 1	
Procesador	Apple M4 (10 núcleos)
Memoria RAM (GB)	16.0 GB
Sistema Operativo	macOS 27.0.1 (arm64)

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	767215.90	3723.55
0.5	438455.27	2986.25
0.7	414779.85	3124.17
0.9	400669.87	3488.32

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	412658.61	2686.87
4.00	400588.52	2746.27
6.00	397036.16	2917.73
8.00	395828.64	3099.23

Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING
- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING

Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf_counter()** en vez de otras funciones como **time.process_time()**?

Porque `time.perf_counter()` mide el tiempo real transcurrido (wall-clock) con el reloj de mayor resolución disponible y es monótonico, es decir, no se ve afectado por cambios en la hora del sistema. Así registra todo lo que tarda la operación tal como la percibe el usuario, incluyendo la lectura de los archivos CSV desde el disco y las esperas del sistema operativo. En cambio, `time.process_time()` solo cuenta el tiempo de CPU que usa el proceso y excluye las esperas de entrada/salida y los tiempos de suspensión, por lo que subestimaría el tiempo real de carga del catálogo. Además, `getTime()` multiplica el valor por 1000 para reportarlo en milisegundos, la unidad que se pide en las tablas.

2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?

`tracemalloc` solo registra las asignaciones de memoria que ocurren mientras el rastreo está activo. `start()` inicia ese rastreo y es lo que permite que `take_snapshot()`, usado en `getMemory()`, capture el estado de la memoria; sin `start()` no es posible tomar la muestra. `stop()` detiene el rastreo y libera las trazas almacenadas. Ambas son importantes porque delimitan exactamente la operación que se quiere medir y porque el rastreo consume memoria y tiempo adicionales en cada asignación: si no se detiene, sigue afectando las operaciones posteriores del programa y las siguientes mediciones. Por esa razón, en la implementación se llama a `tracemalloc.stop()` al terminar la medición, tanto en `load_data()` como en `get_books_by_author_pub_year()`.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

Porque `tracemalloc` intercepta cada asignación de memoria que hace Python para registrar dónde y cuánto se reservó, y ese trabajo adicional hace mucho más lenta la ejecución. Si el tiempo se toma mientras se mide la memoria, el resultado incluye el costo del rastreo y no corresponde al tiempo real de la operación. En nuestras pruebas, la carga del catálogo con LINEAR PROBING y factor de carga 0.5 tomó en promedio 2986.25 ms sin medir memoria y 11059.35 ms midiéndola (cerca de 3.7 veces más); con SEPARATE CHAINING y factor 4.00 pasó de 2746.27 ms a 8669.93 ms (cerca de 3.2 veces más). Por eso el tiempo de ejecución real de las tablas se obtuvo en ejecuciones sin medir memoria y el consumo de datos en ejecuciones separadas. Cada valor de las Tablas 2 y 3 es el promedio de 5 ejecuciones para el tiempo y de 3 ejecuciones para la memoria, usando el archivo `large`.

4. Dado el número de elementos de los archivos (`large`), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

Con el archivo `large` se cargan 10000 libros, 5841 autores distintos, 34252 tags y 999912 registros de `book_tags`, que se agrupan en 34252 llaves (`tag_id`). Por tanto, los índices tienen: `books_by_id` 10000

llaves, books_by_authors 5841, books_by_year_author 5841, tags 34252 y book_tags 34252. El factor de carga es $\alpha = n / M$, es decir, el número de elementos sobre el tamaño de la tabla.

En LINEAR PROBING cada elemento ocupa una posición propia de la tabla, por lo que α siempre debe ser menor que 1; con factores altos se forman grupos de posiciones ocupadas y las búsquedas se alargan. En SEPARATE CHAINING cada posición guarda una lista, por lo que α puede ser mayor que 1 y representa la longitud promedio de las listas.

Como el rehash aumenta la capacidad al siguiente primo del doble cuando se supera el factor límite, el factor de carga real al terminar la carga queda por debajo del límite configurado. Con LINEAR PROBING y límite 0.7, los factores finales fueron: books_by_id 0.44 (10000/22973), books_by_authors y books_by_year_author 0.51 (5841/11483), tags y book_tags 0.37 (34252/91909). Con SEPARATE CHAINING y límite 4.00 fueron: books_by_id 2.46 (10000/4057), books_by_authors y books_by_year_author 2.88 (5841/2027), tags y book_tags 2.11 (34252/16249).

Según los resultados obtenidos, el factor de carga más adecuado para LINEAR PROBING es 0.5, que dio el menor tiempo de carga (2986.25 ms), y para SEPARATE CHAINING está entre 2.00 y 4.00, que dieron los dos menores tiempos (2686.87 ms y 2746.27 ms) con una diferencia de memoria de solo 12070.09 kB.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

En LINEAR PROBING el tiempo no cambia de forma lineal: con 0.1 se obtuvo el mayor tiempo (3723.55 ms), bajó a 2986.25 ms con 0.5 (19.80 % menos) y volvió a subir con 0.7 (3124.17 ms) y con 0.9 (3488.32 ms, 16.81 % más que con 0.5). Con un factor muy bajo las tablas son muy grandes y el costo de crear y hacer rehash de tablas con tantas posiciones, cada una con una entrada vacía, domina el tiempo de carga; con factores altos la tabla está más llena, se forman grupos de posiciones ocupadas y find_slot debe revisar más posiciones antes de encontrar la llave o un espacio disponible.

En SEPARATE CHAINING el tiempo aumenta de forma continua con el factor de carga: 2686.87 ms con 2.00, 2746.27 ms con 4.00, 2917.73 ms con 6.00 y 3099.23 ms con 8.00 (15.35 % más entre 2.00 y 8.00). Un factor mayor implica listas más largas en cada posición de la tabla, y en cada put y get la lista se recorre elemento por elemento hasta encontrar la llave.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

En LINEAR PROBING el consumo de memoria disminuye al aumentar el factor de carga: pasó de 767215.90 kB con 0.1 a 438455.27 kB con 0.5, 414779.85 kB con 0.7 y 400669.87 kB con 0.9 (47.78 % menos entre 0.1 y 0.9). Esto ocurre porque la capacidad de la tabla es aproximadamente n/α y cada posición guarda una entrada (llave, valor) aunque esté vacía; por ejemplo, con 0.1 la tabla de tags llegó a 641261 posiciones para 34252 elementos. El mismo efecto se presenta en los mapas por año de publicación que se crean para cada uno de los 5841 autores. La mayor reducción ocurre entre 0.1 y 0.5; después la diferencia es pequeña (5.40 % entre 0.5 y 0.7 y 3.40 % entre 0.7 y 0.9).

En SEPARATE CHAINING el consumo también disminuye, pero mucho menos: de 412658.61 kB con 2.00 a 395828.64 kB con 8.00 (4.08 % menos), porque al subir el factor solo se reduce el número de

listas de la tabla, mientras que las entradas y los nodos siguen siendo los mismos, uno por cada elemento.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Sí. SEPARATE CHAINING fue más rápido en la carga del catálogo. Su mejor tiempo (2686.87 ms con factor 2.00) es 299.38 ms menor (10.03 %) que el mejor tiempo de LINEAR PROBING (2986.25 ms con 0.5), y el promedio de sus cuatro configuraciones fue 2862.53 ms frente a 3330.57 ms de LINEAR PROBING (14.05 % menos). Incluso su configuración más lenta (3099.23 ms con 8.00) fue ligeramente más rápida que LINEAR PROBING con 0.7 (3124.17 ms). La diferencia se explica porque en SEPARATE CHAINING una colisión solo alarga la lista de la posición correspondiente, mientras que en LINEAR PROBING obliga a buscar posiciones libres consecutivas y los grupos de posiciones ocupadas afectan también a llaves con otros valores de hash. Además, la tabla de LINEAR PROBING siempre tiene más posiciones que elementos, por lo que crearla y hacer rehash implica inicializar más entradas.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Sí, aunque depende del factor de carga. Con factores de carga habituales para cada esquema el consumo es parecido: LINEAR PROBING con 0.7 usó 414779.85 kB y SEPARATE CHAINING con 2.00 usó 412658.61 kB; LINEAR PROBING con 0.9 (400669.87 kB) quedó prácticamente igual que SEPARATE CHAINING con 4.00 (400588.52 kB). La diferencia grande aparece con factores bajos en LINEAR PROBING: con 0.1 consumió 767215.90 kB, cerca de 1.94 veces lo que consumió SEPARATE CHAINING con 8.00 (395828.64 kB), que fue el menor consumo de todas las pruebas. Esto sucede porque en LINEAR PROBING la tabla debe tener más posiciones que elementos ($\alpha < 1$) y cada posición guarda una entrada, mientras que en SEPARATE CHAINING la tabla puede tener menos posiciones que elementos y la memoria crece principalmente con los nodos de las listas, uno por elemento. En ambos esquemas la mayor parte del consumo corresponde a los datos cargados (libros, autores, tags y los 999912 registros de book_tags), que son los mismos en los dos casos; por eso las diferencias entre configuraciones dependen sobre todo del tamaño de las tablas de hash.